

AI Career Coach — Developer Documentation

AI Career Coach — Developer Documentation

Welcome! This doc explains how the AI Career Coach chatbot works, end to end. It assumes no prior context — read top to bottom the first time.

1. What this feature is

A chatbot inside the Messages page that gives BSC users career advice. Users open a chat with the “AI Career Coach” system contact and pick one of two coach personas to talk to:

- **Chataki** — warm, thorough, research-backed tone (default)
- **Botema** — direct, personal, African-tech-context aware

Both personas answer using the same knowledge base and the same underlying code — they differ in system prompt, tone, and the example answers used to keep their voice consistent.

2. Where the code lives

```
src/pages/Messages.tsx          ← frontend chat UI,
persona picker
src/lib/constants.ts            ← AI_COACH_ID
constant

supabase/functions/ai-career-coach/
  index.ts                      ← entry point: request handling, routing,
orchestration
  converser.ts                  ← base framework classes (Converser,
ChatFunction, etc.)
  bsc-coach.ts                  ← Chataki persona (routing instructions +
function list)
  bsc-functions.ts              ← Chataki's functions (calls Azure OpenAI)
  bsc-knowledge.ts              ← the knowledge base (topic → written
coaching content)
  botema-coach.ts               ← Botema persona (routing instructions +
function list + Azure calls)
  botema-examples.ts            ← Botema's real example Q&As (her few-shot
voice data)

supabase/migrations/
  20260525124600_add_ai_coach_profile.sql      ← creates the AI
Coach system user
  20260528000000_allow_system_account_messages.sql ← RLS: lets users
message the AI Coach directly
  20260615100000_converser_tables.sql          ←
coach_wordalisations + coach_user_profiles tables
  20260615110000_seed_wordalisations.sql       ← seed few-shot
examples for Chataki

bsc-coach-training-questions.csv (repo root)    ← question list
used to collect real coach                      answers for
Chataki (see §7)
```

This is a TypeScript/Deno port of a Python framework called the **Converser pattern**, originally built for a different chatbot ("Movivid"/"Otema"). If you want to see the original design reasoning, look at [vivid-insights-main/framework/converser.py](https://github.com/vivid-insights-main/framework/converser.py)

and `vivid-insights-main/.github/instructions/building_conversers_wordalisations.instructions.md` in this repo — they explain the pattern in more depth than this doc does.

3. The Converser pattern, in plain terms

Instead of one big prompt that tries to do everything, the bot's behaviour is split into small **functions**, each with one job. Every message triggers exactly one function. There are four types:

Type	Job	Example
INSTRUCTIONS	Return a fixed, pre-written response	"howCoachWorks" — explains what the coach does
ENGAGE	Return a fixed prompt inviting the user to share more	"inviteUserContext"
CHANGE_CONTEXT	Update some state, then immediately hand off to a <code>WORDALISE</code> function	"updateCareerTopic" sets the topic, then calls "adviseOnCareerTopic"
WORDALISE	The only type that calls the AI. Builds a knowledge-grounded prompt and asks Azure OpenAI to generate the actual reply	"adviseOnCareerTopic", "addressMindsetChallenge"

These are defined as classes in `converser.ts` (`InstructionsFunction` , `EngageFunction` , `ChangeContextFunction` , `WordaliseFunction` — all extend the base `ChatFunction`). Each persona (`BSCCoach` , `BotemaCoach`) implements a small set of concrete functions built on top of these base classes.

Why split it this way? So that only the functions that actually need to generate free-text advice call the AI. Greetings, instructions, and "tell me more" prompts are instant and free — no AI call, no latency, no chance of the AI hallucinating in a place where we want a guaranteed, consistent answer.

4. Request flow, step by step

Entry point: `index.ts`, a single Deno edge function handling `POST /ai-career-coach`.

1. **Frontend sends** `{ message, sender_id, bot }` (`bot` is `"botema"` or `"chataki"`, chosen by the user in the UI — see `Messages.tsx` around the persona-picker dropdown).
2. **Load context**: the function fetches the user's saved profile (`coach_user_profiles`) and their last 10 messages with the AI Coach (`messages` table), then builds a `ConverserContext` (profile + entities + conversation history).
3. **Build the coach**: instantiates `BotemaCoach` or `BSCCoach` (default) with that context.
4. **Route the message**: `routeMessage()` in `index.ts` — a plain regex/keyword classifier, **not an AI call** — decides which function to invoke:
 - Pure greeting → `howCoachWorks`
 - Mindset/emotional language (imposter syndrome, burnout, anxiety, etc.) → `addressMindsetChallenge`
 - Long message with personal background details → `captureUserBackground`
 - Everything else (the common case) → `updateCareerTopic`

Note: the `instructions` getter and `functionSchemas` on each persona class describe an *LLM-based* routing approach (an Azure tool-call that picks the function) — that was the original design, but it was replaced with the faster, free, more predictable keyword router you see in `routeMessage()`. The instructions/schemas are currently unused dead weight from that earlier design; don't be confused if you see them.

5. **Execute the function**: `fn.call(args, message)` runs. If it's a `CHANGE_CONTEXT` function, it updates state and immediately chains into its paired `WORDALISE` function.
6. **Generate the reply** (only for `WORDALISE` functions): builds a prompt from domain knowledge + few-shot examples + recent conversation, sends it to Azure OpenAI, and returns the text.
7. **Save + respond**: the reply is inserted into the `messages` table (as a message from the AI Coach system user) and returned to the frontend as JSON.

5. The knowledge + voice system (WORDALISE functions)

This is the part that actually talks to Azure OpenAI, so it's worth understanding in detail. See `WordaliseFunction.call()` in `converser.ts`:

```
knowledge = getDomainKnowledge(args)           // written coaching
content for the topic
examples = loadFewShotExamples()               // 2-3 real example Q&As
in this coach's voice
prompt    = buildFewShotPrompt(question, knowledge, examples)
reply     = generateResponse(prompt, question)  // → sends to Azure
OpenAI
```

- **Domain knowledge** lives in `bsc-knowledge.ts` as a `KNOWLEDGE_BASE` object keyed by topic (`getting_started` , `cv_job_search` , `mindset` , `salary` , etc.). `classifyTopic()` maps whatever topic string the router extracted onto one of these keys.
- **Few-shot examples** teach the AI *how to sound*, not what to say — they're example answers in the coach's own voice, injected into the prompt so the AI copies the style. This is the "wordalisation" concept — turning structured knowledge into a spoken, personal-sounding answer, in this persona's specific voice.
- The final message sent to Azure is: a system prompt (persona identity/tone) + last 6 messages of conversation history + the few-shot prompt (examples + domain knowledge + the user's question).

Chataki vs Botema — a key current difference

- **Botema's** few-shot examples (`botema-examples.ts`) are drawn from real answers given by an actual BSC coach ("Otema") to a set of career questions. This means Botema is already speaking in her authentic, tested voice.
- **Chataki's** few-shot examples (`coach_wordalisations` DB table, seeded by `20260615110000_seed_wordalisations.sql`) are currently generic placeholder answers, not sourced from a specific real coach yet. `bsc-coach-training-questions.csv` in the repo root is the question list being used to collect real answers for Chataki — once that's filled in and loaded into `coach_wordalisations` , Chataki will get the same kind of authentic voice

Botema already has. Until then, Chataki works correctly but with a more generic tone.

6. Azure OpenAI integration

Both `bsc-functions.ts` and `botema-coach.ts` have their own local `callAzure()` helper that POSTs to:

```
{AZURE_OPENAI_ENDPOINT}openai/deployments/{AZURE_OPENAI_DEPLOYMENT}/chat.  
api-version={AZURE_OPENAI_API_VERSION}
```

Config comes from Supabase secrets (`AZURE_OPENAI_ENDPOINT` , `AZURE_OPENAI_API_KEY` , `AZURE_OPENAI_API_VERSION` , `AZURE_OPENAI_DEPLOYMENT` — currently `gpt-5-nano`), loaded in `index.ts` and passed down as an `AzureConfig` object.

Important gotcha (bit us in production once — see git history around July

2026): `gpt-5-nano` is a *reasoning* model. It spends part of its `max_completion_tokens` budget on hidden reasoning before writing the visible reply. If that budget is set too low, the model can burn the whole thing on reasoning and return **empty content** with a normal 200 OK — no error thrown. Both `callAzure()` helpers currently default to `max_completion_tokens: 2000` and log the `finish_reason` if content ever comes back empty, specifically to make this failure mode visible if it happens again. Don't drop this value back down without testing real (non-trivial) prompts first.

7. Database tables

Table	Purpose
<code>coach_user_profiles</code>	Persists what the coach has learned about a user (career stage, background, target role, goals) across sessions. Written by <code>captureUserBackground</code> , read every time the bot builds a prompt so advice is personalised.
<code>coach_wordalisations</code>	Few-shot example answers for Chataki's <code>WORDALISE</code> functions, keyed by <code>function_name</code> . Service-role-only (RLS blocks direct user access — only the edge function, using the service role key, can read it). This is where Chataki's real voice data (from the training-questions CSV) will eventually live.
<code>messages</code>	The shared messaging table used across the whole app. The AI Coach is a real row in <code>auth.users</code> (id <code>00000000-0000-0000-0000-000000000002</code> , seeded by <code>20260525124600_add_ai_coach_profile.sql</code>) so normal foreign keys and RLS policies work without special-casing. A dedicated RLS policy (<code>20260528000000_allow_system_account_messages.sql</code>) lets any authenticated user message this system account directly, without needing an existing pairing/booking.

8. Frontend integration (`src/pages/Messages.tsx`)

- `selectedBot` state (`"botema" | "chataki" | null`), persisted in `sessionStorage` so the choice survives a page refresh within the session.
- Persona picker dropdown lets the user switch between Botema and Chataki at any time.
- Sending a message to the AI Coach contact calls the edge function:

```
supabase.functions.invoke('ai-career-coach', {
  body: { message: msgText, sender_id: user.id, bot: selectedBot ??
    "chataki" }
})
```

- The function's reply is written straight to the `messages` table by the edge function itself (step 7 above) — the frontend just re-fetches messages afterwards rather than rendering the HTTP response directly.

9. How to extend this

Add a new topic to the knowledge base: add an entry to `KNOWLEDGE_BASE` in `bsc-knowledge.ts`, and add a matching pattern in `classifyTopic()` so the router can reach it.

Add a new function (new type of thing the coach can do): create a class extending the right base (`WordaliseFunction` if it needs to generate free text, `InstructionsFunction` / `EngageFunction` for fixed responses, `ChangeContextFunction` if it needs to update state first), add it to both personas' `initializeFunctions()` if it should exist in both, and add a routing rule for it in `routeMessage()` in `index.ts`.

Add/improve a persona's voice: for Botema, add more real Q&A pairs to `BOTEMA_EXAMPLES` in `botema-examples.ts`. For Chataki, add rows to `coach_wordalisations` (via a migration, matching the pattern in `20260615110000_seed_wordalisations.sql`) — ideally sourced from a real coach's answers, same as Botema.

Add a third persona: create a new `*-coach.ts` file mirroring `botema-coach.ts` (own system prompt, own `callAzure`, own function set), wire it into the `bot` parameter switch in `index.ts`, and add it to the picker dropdown in `Messages.tsx`.

10. Deploying and debugging

Deploy: `npx supabase functions deploy ai-career-coach` (requires the project to be linked and Docker running — you'll get a warning if Docker isn't running, but the deploy still works since these are simple Deno functions).

Debugging an empty/broken reply: 1. Check the edge function logs in the Supabase dashboard (Functions → ai-career-coach → Logs) for `console.error` output — both `callAzure()` helpers log the raw Azure error body on a non-OK response, and log `finish_reason` if content came back empty. 2. If `finish_reason` is `"length"`, the token budget was exhausted — see §6. 3. If the error is about missing credentials,

check `npx supabase secrets list` — the four `AZURE_OPENAI_*` secrets must all be set. 4. `console.log('[Converser] Routing → ...')` in `index.ts` shows which function the keyword router picked for a given message — useful for figuring out if a message was misrouted rather than the AI call actually failing.